BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 3

Roll no: 41 PRN No: 12410751 Name of Student: Rucha Narwade

Subject Name : Deep Learning

Problem Statement

Implement Forward Propagation and Backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm

1. Start the program.
2. Import the required libraries: TensorFlow, Keras, Matplotlib.
3. Load the MNIST handwritten digit dataset using mnist.load_data().
4. Normalize the training and testing images by dividing pixel values by 255.
5. Create a Sequential Neural Network model.
6. Add a Flatten layer to convert 28×28 images into a 1D vector.
7. Add a hidden Dense layer with 128 neurons using ReLU activation.
8. Add an output Dense layer with 10 neurons using Softmax activation.
9. Compile the model using the Adam optimizer with a learning rate of 0.001 and Sparse Categorical Crossentropy loss function.
10. Train the model for 5 epochs using the training dataset.
11. Plot the training and validation accuracy graph.
12. Plot the training and validation loss graph.
13. Evaluate the trained model using the testing dataset.
14. Display the testing loss and testing accuracy.

15. Train the model using different learning rates (0.1, 0.01, 0.001, 0.0001) and compare the accuracies.
16. Train the model using different epochs (1, 5, 10, 20) and compare the accuracies.
17. Analyze the effect of learning rates and epochs on model performance.
18. Stop the program.

Code

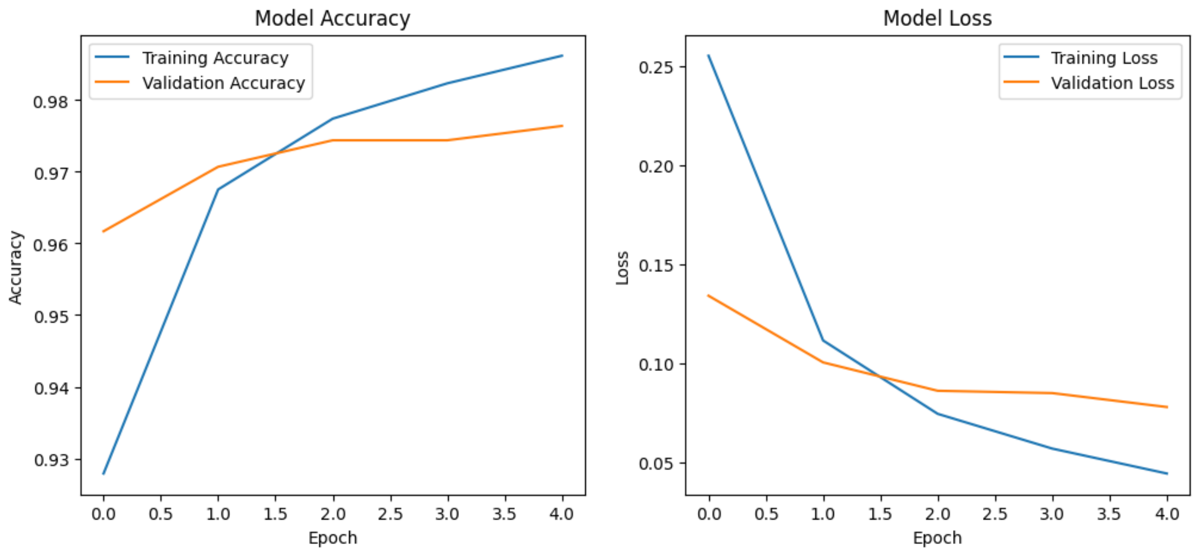
Google Colab:

https://colab.research.google.com/drive/1IiRv3jo-Tl7L6m5xKPeUi0kG-_dI8EFB?usp=sharing

---

Output

Training Accuracy Graph and Training Loss Graph



Backpropogation:

```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_data=(x_test,y_test)
)
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9279 - loss: 0.2551 - val_accuracy: 0.9617 - val_loss: 0.1341
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9675 - loss: 0.1115 - val_accuracy: 0.9707 - val_loss: 0.1005
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9774 - loss: 0.0745 - val_accuracy: 0.9744 - val_loss: 0.0861
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9823 - loss: 0.0570 - val_accuracy: 0.9744 - val_loss: 0.0850
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9862 - loss: 0.0444 - val_accuracy: 0.9764 - val_loss: 0.0780
```

Example:

Test Loss : 0.0785

Test Accuracy : 97.85%

Learning Rate Analysis

Training with Learning Rate = 0.1

Accuracy: 0.5432999730110168

Training with Learning Rate = 0.01

Accuracy: 0.9632999897003174

Training with Learning Rate = 0.001

Accuracy: 0.9778000116348267

Training with Learning Rate = 0.0001

Accuracy: 0.9502000212669373

Epoch Analysis

Training with Epochs = 1

Accuracy: 0.95660001039505

Training with Epochs = 5

Accuracy: 0.9764000177383423

Training with Epochs = 10

Accuracy: 0.977400004863739

Training with Epochs = 20

Accuracy: 0.9796000123023987